

实验二：原生应用开发（.NET MAUI 技术栈）

一、实验基本信息

- 实验编号：d20301035102、d20301009202
- 学时分配：4 学时
- 实验类型：验证型实验
- 每组人数：6 人（分组协作完成）
- 对应课程目标：课程目标 1（掌握原生应用开发基础）

二、实验目的

1. 掌握 .NET MAUI 原生应用的核心开发流程与技术要点，熟悉 C# + XAML 的开发模式。
2. 掌握 .NET MAUI 页面布局、控件使用（Entry、Label、Button 等）与事件绑定方法。
3. 掌握 .NET MAUI 页面导航机制与页面间数据传递技巧（构造函数传参）。
4. 掌握输入验证、异常处理的基本方法，确保应用程序的健壮性。
5. 理解 .NET MAUI 与 Android、iOS 原生开发的差异特点，明确其跨平台原生开发优势。
6. 能够运用 AI 辅助工具进行代码生成、错误排查与代码优化，提升开发效率。

三、实验环境

3.1 .NET MAUI 开发环境

- 开发工具：Visual Studio 2022（需安装 MAUI 开发工具）
- 开发语言：C#

- UI 框架：XAML / MAUI 原生组件
- 框架版本：.NET 8
- 调试设备：Android 模拟器、iOS 模拟器或 Windows 机器
- 辅助工具：TRAIE AI 辅助编码工具

四、实验内容

4.1 核心任务

使用 .NET MAUI (C# + XAML) 实现跨平台原生登录页面，完成以下统一功能：

1. 实现学号、密码输入功能，支持学号数字输入、密码隐藏显示。
2. 完成输入验证（不能为空、文本去空格处理），添加错误提示机制。
3. 实现登录按钮点击事件，完成模拟登录验证逻辑。
4. 实现页面跳转与数据传递，登录成功跳转至个人中心并显示个性化信息。
5. 完成组内技术对比，分析 .NET MAUI 与 Android、iOS 原生开发的差异。

4.2 实战案例

开发“智慧校园”登录模块：学生在登录页面输入学号和密码，系统验证成功后跳转至个人中心页面，显示个性化的欢迎信息和用户资料；验证失败则给出明确错误提示，保证程序健壮性。

测试账号信息

```
plain text
测试账号：admin
测试密码：123456
```

五、实验步骤

5.1 .NET MAUI 登录页面开发

1. ****需求分析****：明确功能需求（输入、验证、跳转、提示）与界面需求（原生风格、布局合理、交互友好）。
2. ****创建项目****：打开 Visual Studio 2022，新建“.NET MAUI App”项目，命名为

NativeLogin，选择 .NET 8 框架，等待依赖项加载完成。

- AI 辅助编码**：向 TRAE 描述需求，获取 XAML 布局与 C# 后台逻辑基础代码，手动完善业务细节。
- 登录页面布局 (LoginPage.xaml)**：使用 VerticalStackLayout 布局，添加 Entry（学号、密码输入框）、Label（错误提示）、Button（登录按钮），配置控件属性。
- 登录页面逻辑 (LoginPage.xaml.cs)**：编写登录按钮点击事件，实现输入去空格、空值验证、模拟登录验证、页面跳转与数据传递。

5.2 .NET MAUI 个人中心页面开发

- 创建页面**：在项目中新建 ContentPage，命名为 HomePage，作为个人中心页面。
- 个人中心布局 (HomePage.xaml)**：使用 VerticalStackLayout 布局，添加 Label 控件，用于显示页面标题和个性化欢迎信息。
- 个人中心逻辑 (HomePage.xaml.cs)**：编写构造函数，接收登录页面传递的用户名参数，绑定到欢迎信息 Label。
- 配置程序入口**：修改 App.xaml.cs，将登录页面设置为启动页，并添加 NavigationPage 容器，支持页面跳转。

5.3 项目调试与平台对比

- 项目调试**：选择调试设备（Android 模拟器、iOS 模拟器或 Windows 机器），启动调试，测试空输入验证、错误账号验证、正确账号跳转、数据显示等核心功能，调试并修复异常问题。
- 平台差异对比**：组内协作，对比 .NET MAUI 与 Android、iOS 原生开发在开发语言、UI 框架、页面跳转、数据传递等方面的差异，完成技术对分享。

六、实验结果

6.1 成功标准验证

成功标准	验证结果
登录页面功能完整	✓ 完成学号、密码输入，支持密码隐藏，布局规范

输入验证功能正常	✔ 空输入、空格输入均能正确验证，错误提示清晰
页面跳转与数据传递正常	✔ 登录成功跳转至个人中心，正确显示欢迎信息
调试无异常	✔ 无空引用、跳转异常等问题，程序运行稳定
完成平台差异对比	✔ 明确 .NET MAUI 与 Android、iOS 原生开发差异，完成分享

6.2 界面特点

- .NET MAUI 登录页面：**遵循 MAUI 原生风格，控件对齐规范、布局合理，输入框占位提示清晰，密码输入框默认隐藏，错误提示使用红色文本居中显示，交互友好，适配多平台显示。
- .NET MAUI 个人中心页面：**布局简洁，标题突出，欢迎信息清晰，贴合“智慧校园”主题，适配不同设备屏幕尺寸。

6.3 平台对比分析（精简版）

对比项	Android（原生）	iOS（原生）	.NET MAUI
开发语言	Kotlin	Swift	C#
UI 框架	XML/Compose	SwiftUI	XAML
页面跳转	Intent	NavigationLink	Navigation.PushAsync
跨平台支持	仅安卓	仅 iOS	多端运行（安卓、iOS、Windows 等）

6.4 应用架构图（文字版）

plain text
界面层 → 逻辑层 → 导航层

XAML 布局（LoginPage/HomePage）→ C#后台逻辑（验证、传参）→ Navigation 导航容器

七、平台/技术栈对比分析

7.1 开发语言对比

技术栈	开发语言	语言特点	学习曲线
Android 原生	Kotlin	强类型、面向对象、简洁高效	中等
iOS 原生	Swift	强类型、安全高效、语法简洁	中等
.NET MAUI	C#	强类型、.NET 生态完善、面向对象	中等

7.2 开发效率对比

技术栈	开发效率	调试体验	跨端优势
Android 原生	中等	良好	无
iOS 原生	中等	良好	无
.NET MAUI	高	优秀	一次编写，多端运行

7.3 生态与支持对比

技术栈	社区活跃度	第三方库	官方支持
Android 原生	非常活跃	丰富	Google
iOS 原生	非常活跃	丰富	Apple

.NET MAUI	活跃	较丰富	Microsoft
-----------	----	-----	-----------

7.4 适用场景分析

- 1. **Android 原生**：仅需开发安卓平台应用、追求安卓平台极致原生体验的项目。
- 2. **iOS 原生**：仅需开发 iOS 平台应用、追求 iOS 平台极致原生体验的项目。
- 3. **.NET MAUI**：.NET 技术栈团队、需要同时开发多平台（安卓、iOS、Windows）原生应用、追求开发效率与跨端一致性的项目。

八、实验总结

8.1 技术收获

- 1. 掌握 .NET MAUI 原生应用的开发流程，熟悉 XAML 布局控件（Entry、Label、Button 等）的使用方法。
- 2. 掌握 C# 后台逻辑编写，理解事件绑定、输入验证、页面跳转与数据传递的核心原理。
- 3. 掌握 .NET MAUI 导航容器的配置方法，解决页面跳转异常等常见问题。
- 4. 学会使用 AI 辅助工具进行代码生成、错误排查与代码优化，提升开发效率。
- 5. 理解 .NET MAUI 与 Android、iOS 原生开发的差异，明确其跨平台原生开发的优势与特点。

8.2 项目结构

```
plain text
NativeLogin/
├─ LoginPage.xaml           登录页面布局
├─ LoginPage.xaml.cs       登录页面后台逻辑
├─ HomePage.xaml           个人中心页面布局
├─ HomePage.xaml.cs        个人中心页面后台逻辑
└─ App.xaml.cs             程序入口与导航配置
```

8.3 技术栈选择建议

- 仅开发安卓原生应用：选用 Android（Kotlin）原生开发

- 仅开发 iOS 原生应用：选用 iOS (Swift) 原生开发
- .NET 技术栈、需多平台原生应用：选用 .NET MAUI

8.4 后续学习建议

1. 深入学习 .NET MAUI 高级特性，如 MVVM 架构、数据绑定进阶、原生能力调用。
2. 掌握 .NET MAUI 跨平台适配技巧，解决不同设备屏幕尺寸、系统版本的适配问题。
3. 学习本地存储、权限管理、网络请求等进阶功能，完善应用功能。
4. 深入研究 Android、iOS 原生开发，提升跨平台与原生融合开发能力。
5. 学习自动化测试、应用打包与发布流程，提升项目交付能力。

九、思考与讨论

9.1 原生开发与跨平台原生开发优缺点

原生开发 (Android/iOS) 优点：

- 平台适配性最好，能充分调用平台特有功能，用户体验极致
- 性能最优，无跨平台桥接损耗
- 生态成熟，第三方库丰富，问题解决方案多

原生开发 (Android/iOS) 缺点：

- 需分别开发安卓、iOS 版本，开发与维护成本高
- 需掌握两种不同开发语言与技术栈，学习成本高
- 迭代速度慢，需分别对两个平台进行更新迭代

.NET MAUI 跨平台原生开发优点：

- 一次编写，多端运行，显著降低开发与维护成本
- 基于 .NET 生态，技术栈统一，便于团队协作
- 原生渲染，性能接近原生开发，用户体验良好

.NET MAUI 跨平台原生开发缺点：

- 平台特有功能调用需额外适配，存在一定限制
- 生态成熟度略低于原生开发，部分场景第三方库支持不足

- 调试复杂度相对较高，多平台异常排查难度大

9.2 技术栈选型思路

1. 明确目标平台（单一平台/多平台）、性能要求与功能复杂度
2. 结合团队现有技术栈与学习成本，避免技术栈跨度过大
3. 评估开发周期、预算与长期维护成本，优先选择高效、易维护的技术栈
4. 验证技术栈生态、社区支持与第三方库资源，确保问题可解决
5. 开发原型进行技术验证，对比不同技术栈的开发效率与运行效果

9.3 原生应用开发趋势

1. 跨平台原生开发成为主流，.NET MAUI、Flutter 等框架持续优化，逐步弥补与纯原生开发的差距
2. AI 辅助开发普及，降低开发门槛，提升开发效率，减少重复编码工作
3. 原生与跨平台融合开发成为趋势，兼顾开发效率与用户体验
4. 开发工具链持续完善，调试、打包、发布流程更加便捷高效

实验完成日期：2026 年 4 月 20 日

实验者：李世程

学号：2020211596